

Coding & Solution

Date	29 June 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Geethika-27/Smart-Lender
Programming Language(s)	Python, JavaScript, HTML, CSS
Framework(s) Used	Flask (v3.1.3), Bootstrap (v5.3.0), Scikit-Learn (>=1.2.0), XGBoost (>=1.7.0)
Key Features Implemented	• Interactive Web UI: High-fidelity, responsive, banking-themed client interface with integrated Light/Dark mode toggles backed by browser session persistence. • Dynamic UI Mechanics: A real-time form completion progress bar, live applicant loan-to-income annual burden calculator, and a live client-side profile warning/hint engine. • Form Preset & Draft Features: One-click automated scenario preset buttons (Low-Risk, High-Risk, and Clear Form) and browser LocalStorage-based draft save/load capabilities. • Two-Tier Data Validation: Strict client-side HTML5 input validation and server-side type-casting, range checks, and categorical checks in app.py against predefined field rules. • ML Inference Pipeline: Pickled model pipeline loading a trained XGBoost Classifier (rdf.pkl with ~82% test accuracy) and feature scaling parameters (scale.pkl) to serve instantaneous real-time classification.

	• Advanced Results Analytics: Approval/Rejection output accompanied by a probability-derived confidence percentage and risk band categorization ("Low Risk", "Moderate Risk", "High Risk"). • Cloud-Ready Configurations: Completed deployment manifests and assets (Dockerfile, manifest.yml, Procfile, runtime.txt) optimized for IBM Cloud Code Engine or Cloud Foundry deployment.
Pending / Incomplete Features	• Remote Version Control Deployment: Uploading the local project directory and tracking history onto a remote repository hosting service (GitHub or GitLab). • Relational Database Integration: Integrating SQLite or PostgreSQL persistence to save historical loan records and decision logs (currently relies on local browser storage drafts). • User Authentication & Security: Implementing a secure login/registration system for applicants and bank loan officers (RBAC). • Analytics Dashboard: Implementing frontend interactive data charts (e.g., Chart.js) to display predictive distribution rates and model validation metrics.
Setup / Run Instructions	1. Open your terminal and navigate to the project root directory. 2. Install the required dependencies: pip install -r requirements.txt 3. Run the machine learning pipeline (Jupyter Notebook) to train and export serialized models: Training/Loan_Prediction_using_ML.ipynb 4. Launch the local Flask application server: cd Flask python app.py 5. Open a web browser and navigate to: http://127.0.0.1:5000

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	No

Additional Notes / Comments:

- **Modularity:** The Python backend logic (app.py) is decoupled from frontend pages. The training notebook contains structured modular functions (e.g., decisionTree(), RandomForest(), KNN(), XGB()) to isolate distinct training paths.
- **Variable Naming:** Code follows strict naming patterns matching dataset features (e.g., ApplicantIncome, CoapplicantIncome, Credit_History) and readable function identifiers (e.g., parse_and_validate_form(), updateBurden(), updateLiveHint()).
- **Error Handling:** Form inputs are safely parsed via try-except blocks, handling data formatting errors (ValueError) and general server exceptions gracefully by returning standard error-state views instead of failing silently.
- **Version Control Status:** The repository has been set up locally, but the code has not yet been pushed to a remote hosting platform (GitHub/GitLab). Once you upload the project, you can update the link in the table and flip the checklist item #6 to Yes.